

✓ Lab 4 Report:

Surpass Human Performance in Fashion MNIST Classification

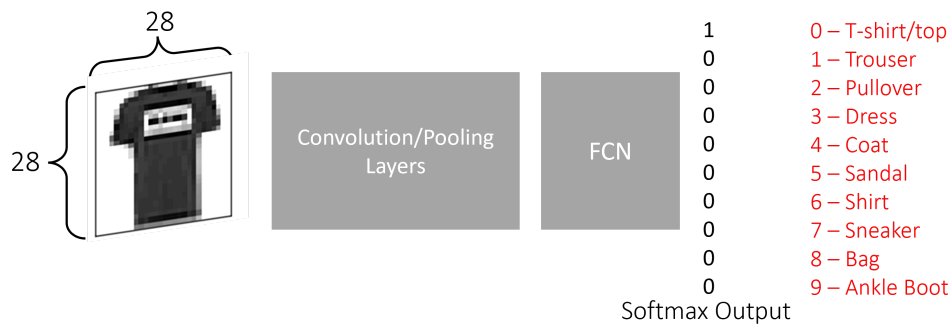
✓ Name: Chloe Jin

```
%matplotlib inline
import matplotlib.pyplot as plt
import torch
import numpy as np
```

```
from IPython.display import Image # For displaying images in colab jupyter cell
```

```
Image('lab4_exercise.png', width = 1000)
```

Surpass Human Performance in Fashion MNIST Classification



In this exercise, you will classify fashion item class (28 x 28) using your own **Convolutional Neural Network Architecture**.

Prior to training your neural net, 1) Normalize the dataset using standard scaler and 2) Split the dataset into train/validation/test.

Design your own CNN architecture with your choice of Convolution/Pooling/FCN layers, activation functions, optimization method etc.

Your goal is to **achieve a testing accuracy of >89%**, with no restrictions on epochs (**Human performance: 83.5%**).

Demonstrate the performance of your model via plotting the **training loss**, **validation accuracy** and printing out the **testing accuracy**.

After your model has reached the goal, print the accuracy in each class. What is the class that your model performed the worst?

44

✓ Prepare Data

```
# Load Fashion-MNIST Dataset in Numpy

# 10000 training features/targets where each feature is a greyscale image with shape (28, 28)
train_features = np.load('fashion_mnist_train_features.npy')
train_targets = np.load('fashion_mnist_train_targets.npy')

# 1000 testing features/targets
test_features = np.load('fashion_mnist_test_features.npy')
test_targets = np.load('fashion_mnist_test_targets.npy')

# Let's see the shapes of training/testing datasets
```

```
print("Training Features Shape: ", train_features.shape)
print("Training Targets Shape: ", train_targets.shape)
print("Testing Features Shape: ", test_features.shape)
print("Testing Targets Shape: ", test_targets.shape)
```

```
Training Features Shape: (10000, 28, 28)
Training Targets Shape: (10000,)
Testing Features Shape: (1000, 28, 28)
Testing Targets Shape: (1000,)
```

```
# Visualizing the first three training features (samples)
```

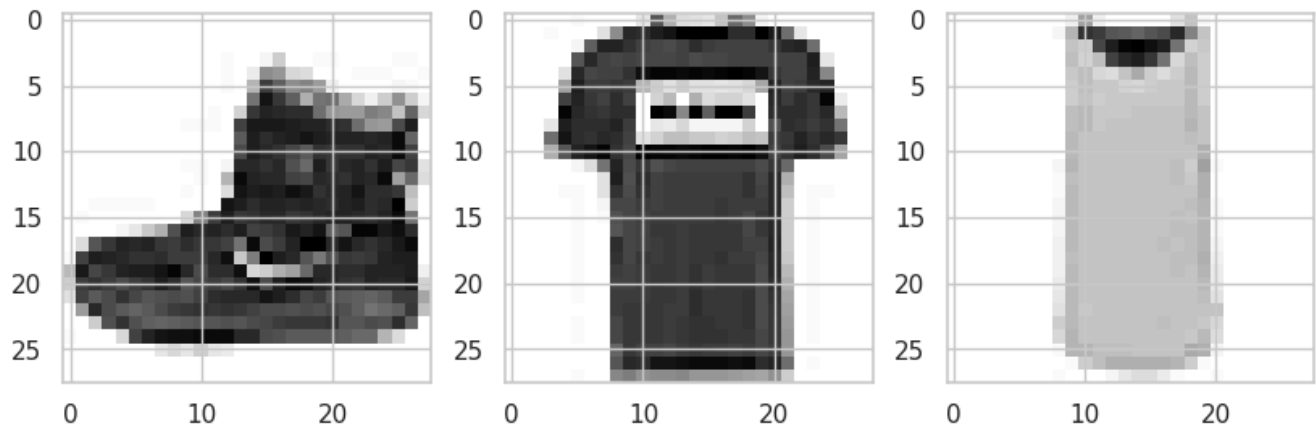
```
plt.figure(figsize = (10, 10))

plt.subplot(1,3,1)
plt.imshow(train_features[0], cmap = 'Greys')

plt.subplot(1,3,2)
plt.imshow(train_features[1], cmap = 'Greys')

plt.subplot(1,3,3)
plt.imshow(train_features[2], cmap = 'Greys')
```

```
<matplotlib.image.AxesImage at 0x78b89dc31cd0>
```



```
train_features.shape[0]
```

```
10000
```

```
# Reshape features via flattening the images
# This refers to reshape each sample from a 2d array to a 1d array.
# hint: np.reshape() function could be useful here
```

```
train_features = np.reshape(train_features, (train_features.shape[0], -1))
test_features = np.reshape(test_features, (test_features.shape[0], -1))
train_targets.shape
```

```
(10000,)
```

```
# Define your scaling function
def scaling(x):
    return (x - np.mean(x))/np.std(x)
```

```
# Scale the dataset according to standard scaling
train_features = scaling(train_features)
test_features = scaling(test_features)
```

```
# Take the first 1000 (or randomly select 1000) training features and targets as validation set
```

```
validation_features = train_features[ : 1000, :]
validation_targets = train_targets [ : 1000, ]

# Take the remaining 9000 training features and targets as training set

train_features = train_features [1000 :, :]
train_targets = train_targets [1000 :, ]
```

```
# Reshape train/validation/test sets to conform to PyTorch's (N, Channels, Height, Width) standard for C

train_features = train_features.reshape(-1, 1, 28, 28)
validation_features = validation_features.reshape(-1, 1, 28, 28)
test_features = test_features.reshape(-1, 1, 28, 28)
train_features.shape

(9000, 1, 28, 28)
```

✓ Define Model

```
# Define your CNN architecture here

class CNNModel(torch.nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()

        # Convolution 1
        self.cnn1 = torch.nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=0)
        self.relu1 = torch.nn.ReLU() # activation function 1

        # Max pool 1
        self.maxpool1 = torch.nn.MaxPool2d(kernel_size=2)

        # Convolution 2
        self.cnn2 = torch.nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=0)
        self.relu2 = torch.nn.ReLU() # activation function 2

        # Max pool 2
        self.maxpool2 = torch.nn.MaxPool2d(kernel_size=2)

        # Fully connected 1
        self.fc1 = torch.nn.Linear(64 * 5 * 5, 128)
        self.relu3 = torch.nn.ReLU() # activation function 3

        # Fully connected 2
        self.fc2 = torch.nn.Linear(128, 10)

    def forward(self, x):
        # Input x dimensions: # nx1x28x28
        # Set 1
        out = self.cnn1(x) # nx32x26x26
        out = self.relu1(out)
        out = self.maxpool1(out) # nx32x13x13

        # Set 2
        out = self.cnn2(out) # nx64x11x11
        out = self.relu2(out)
        out = self.maxpool2(out) # nx64x5x5

        # Flatten
        out = out.view(out.size(0), -1) # nx800

        # Fully connected
        out = self.fc1(out) # nx128
```

```
out = self.relu3(out)
out = self.fc2(out) # nx10
return out
```

✓ Select Hyperparameters

```
# Fix the random seed so that model performance is reproducible
torch.manual_seed(55)

# Initialize your CNN model

model = CNNModel()

# Define learning rate, epoch and batchsize for mini-batch gradient

learning_rate = 0.001
epochs = 30
batchsize = 70

# Define loss function and optimizer
# Cross Entropy Loss
loss_func = torch.nn.CrossEntropyLoss()

# Adam Optimizer
optimizer = torch.optim.Adam(model.parameters(), lr = learning_rate)

model

CNNModel(
  (cnn1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1))
  (relu1): ReLU()
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (cnn2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
  (relu2): ReLU()
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=1600, out_features=128, bias=True)
  (relu3): ReLU()
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)
```

✓ Identify Tracked Values

```
# Placeholders for training loss and validation accuracy during training
# Training loss should be tracked for each iteration (1 iteration -> single forward pass to the network)
# Validation accuracy should be evaluated every 'Epoch' (1 epoch -> full training dataset)
# If using batch gradient, 1 iteration = 1 epoch

train_loss_list = []
validation_accuracy_list = []
```

✓ Train Model

```
import torch
from torch.utils.data import TensorDataset, DataLoader
import tqdm
import numpy as np

# Use available GPU
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

```

# Convert training, validation, test dataset into torch format
train_features = torch.tensor(train_features).float()
train_targets = torch.tensor(train_targets).long()

validation_features = torch.tensor(validation_features).float()
validation_targets = torch.tensor(validation_targets).long()

test_features = torch.tensor(test_features).float()
test_targets = torch.tensor(test_targets).long()

# Pairs inputs and labels together into a dataset object
train_dataset = TensorDataset(train_features, train_targets)

# User DataLoader iterator would improve a lot of accuracy
train_loader = DataLoader(
    train_dataset,
    batch_size=batchsize,
    shuffle=True, # Shuffles it to reduce overfitting
    pin_memory=True if device.type == "cuda" else False # Use GPU
)

# Move model to GPU
model = model.to(device)

# Loop through iterating epochs
for epoch in tqdm.trange(epochs):

    # Start training
    model.train()
    epoch_loss = 0.0 # Preset epoch loss

    for inputs, targets in train_loader:

        # Move inputs/targets dataset to GPU
        inputs = inputs.to(device, non_blocking=True)
        targets = targets.to(device, non_blocking=True)

        # Clear gradients
        optimizer.zero_grad()

        # Forward Propagation
        outputs = model(inputs)

        # Calculate softmax and cross entropy loss
        loss = loss_func(outputs, targets)

        # Calculate gradients
        loss.backward()

        # Update parameters
        optimizer.step()

        # Evaluate accuracy using predictions
        preds = torch.argmax(outputs, dim=1)
        accuracy = (preds == targets).float().mean().item()
        train_loss_list.append(loss.item()) # Add loss into list

    # Evaluate accuracy
    model.eval()
    with torch.no_grad():
        outputs = model(validation_features.to(device))
        preds = torch.argmax(outputs, dim=1)

    # Calculate accuracy using predicted and actual targets

```

```
acc = (preds == validation_targets.to(device)).float().mean().item()
```

```
# Add values in the list  
validation_accuracy_list.append(acc)
```

```
100%|██████████| 30/30 [04:08<00:00, 8.28s/it]
```

Visualize & Evaluate Model

```
# Seaborn for prettier plot
```

```
import seaborn as sns
```

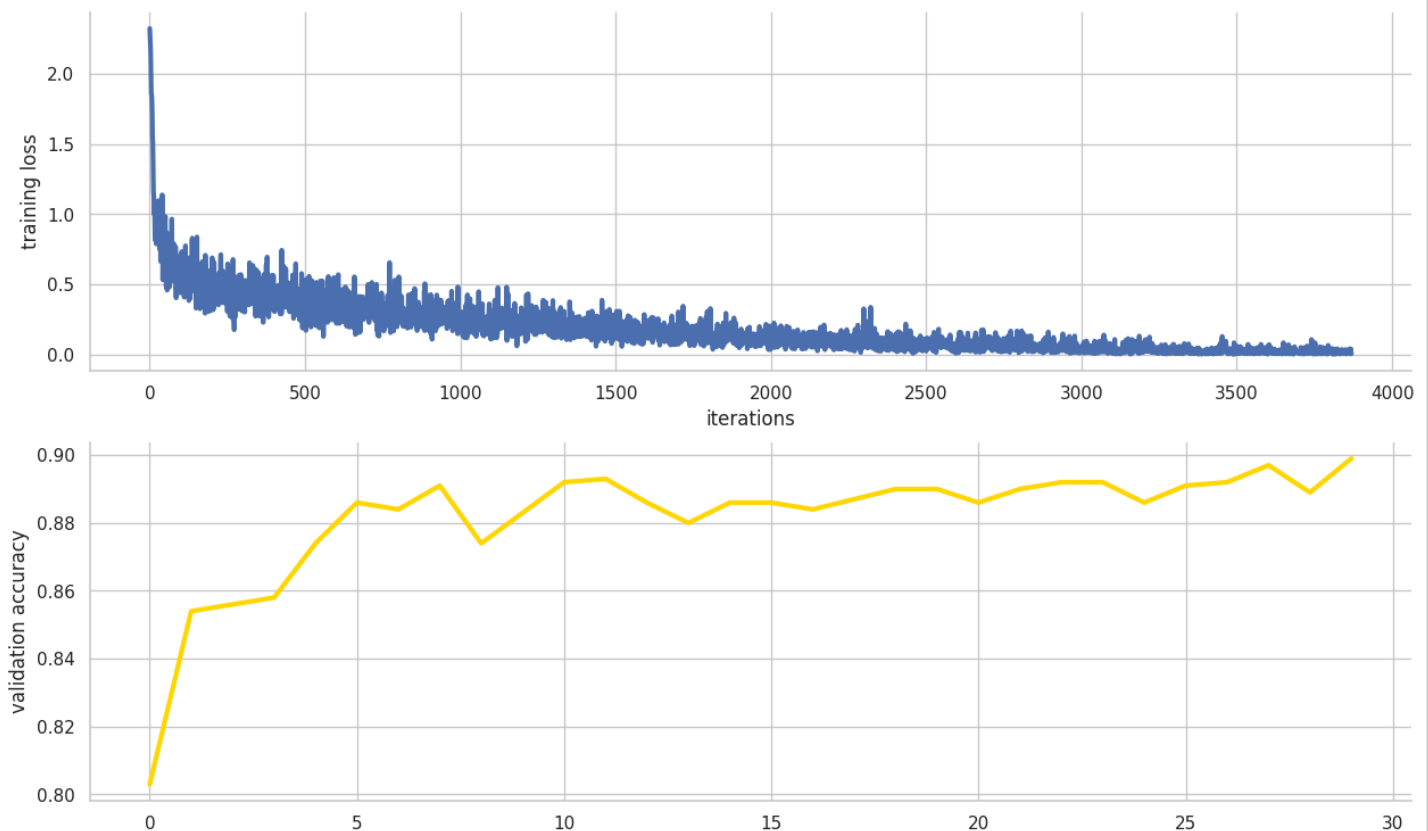
```
sns.set(style = 'whitegrid', font_scale = 1)
```

```
# Visualize training loss
```

```
plt.figure(figsize = (15, 9))
```

```
plt.subplot(2, 1, 1)  
plt.plot(train_loss_list, linewidth = 3)  
plt.ylabel("training loss")  
plt.xlabel("iterations")  
sns.despine()
```

```
plt.subplot(2, 1, 2)  
plt.plot(validation_accuracy_list, linewidth = 3, color = 'gold')  
plt.ylabel("validation accuracy")  
sns.despine()
```



```
model.eval()  
with torch.no_grad():  
    test_outputs = model(test_features.to(device))
```

```
test_preds = torch.argmax(test_outputs, dim=1)
test_acc = (test_preds == test_targets.to(device)).float().mean().item()
```

```
print("Test Accuracy:", test_acc)
```

Test Accuracy: 0.8840000033378601

开始借助 AI 编写或生成代码。